

PlateletIQ — Team Project Status & Implementation Report

Complete Technical Summary & Operational Feature Audit (Ready for Evaluation)

Hackathon: DSU DevHack 3.0 | Facility: Govt. General Hospital Chennai (TN-GGH-001) | Date: Sep 18, 2026

1. Executive Summary & Accomplishments

PlateletIQ is an end-to-end, AI-driven platelet demand forecasting, inventory management, and regional transfer platform. **Every screen in the application is fully integrated with live backend REST APIs** and built strictly according to Apple Human Interface Guidelines (HIG). All static fallback objects (`{| { ... }}`) have been completely eliminated from the codebase.

Component	Implementation Status	Verification Result
Frontend UI (React 19 + Vite 8)	10/10 Screens Built, Pixel-Matched to HTML Mockups	PASSED (0 Build Errors)
Backend API (FastAPI + Py 3.12)	9 HTTP Routers, Pydantic v2 Schemas, OpenAPI Docs (/docs)	PASSED (0 Syntax Errors)
ML Forecasting Core	scikit-learn LASSO Engine (MASE 0.734 vs Naive 0.993)	PASSED (Live Inferred)
Database & Seeding	SQLite (plateletiq.db) + eRaktKosh Scraper DB (eraktkosh.db)	PASSED (Seeded & Active)
Interactive Workflows	All Drawers, Buttons, Modals & API Mutations Connected	PASSED (100% Functional)

2. Screen-by-Screen Feature & Workflow Audit

Screen	Key Features & UI Enhancements	Interactive Actions & API Integration
1. Overview (Mockup 1/3)	• 4-Column Stat Cards (Available 48, Expected 26 u/d, Horizon 7d, At Risk 9 units / ■27,000 in red) • Hero Recommendation Card with left amber stripe • Transfer Opportunity Card (Bangalore → Chennai) • Demand Chart (Solid blue actuals vs dashed forecast)	• HOLD — Stock Healthy or PROCURE 18 units display • Interactive Why → button opens Decision Engine evidence drawer • Interactive Review Transfer → button opens transfer drawer
2. Forecast (Mockup 2/3)	• Chart-first layout with inline badge (LASSO v1.4 · MASE 0.734) • 3-Layer Quantile Chart (q50 dots, q67 band, q90 buffer, weekend shading) • 7-Column Demand Prediction Heatmap • 2 Visual Insight Cards (Weekly Pattern, Training Data 4,018d)	• Interactive 7d / 14d / 30d horizon pill toggle • Custom chart hover tooltip • Live ML inference via forecastApi.getLatest
3. Inventory	• Proportional Shelf-Life Strip (5-day expiry window) • FIFO-sorted unit directory with realistic IDs (RDP-8001, SDP-8002) • Component & blood group filters	• + Register Unit drawer submits inventoryApi.registerUnit • Issue Unit → modal enforces FIFO and submits issueUnit
4. Actions	• Top Transfer Opportunity alert card • Recommendation card with 36px bold action text • Projected gap & inventory surplus metrics • 24h Expiry Watch list	• Review Action → opens evidence drawer • Confirm Recommendation posts to backend audit log • Adjust modal validates mandatory reason • Review Transfer → submits inter-bank offer
5. Data Setup	• CSV Drag-and-Drop / Manual 30-day time-series entry • Sequential 5-Step Pipeline animation reveal • Instant 7-day mini forecast heatmap on completion	• forecastApi.uploadHistory executes ML pipeline on backend • Input row validation (<14 rows error message)
6. Analytics (Mockup 3/3)	• 3 Hero Cards (Wastage 3.25%, MASE 0.734, Training 4,018d) • Horizontal Wastage Reduction comparison bars • ■ Saves ■7.2 lakh/year callout • MASE Benchmark comparison bars vs Schilling • 4-Tile Model Transparency grid + status pills	• Connected to analyticsApi.getForecastAnalytics • Ground Truth numbers strictly enforced • Framer motion bar growth animations
7. Transfers & Recovery	• Regional transfer opportunity list • Urgency-ranked at-risk unit list by pathway (Clinical, Transfer, Research/HPL, Discard)	• Review → button opens transfer review drawer • Submit Transfer Offer posts transferApi.makeOffer
8. Network	• SDP Scarcity Alert card (4.7% stock, 23/32 districts zero SDP) • Data Quality & Staleness summary card • District stock table with freshness dots	• Reads real scraped 35MB eraktkosh.db SQLite dataset • Sorts districts by total stock dynamically

9. Waste Recovery	• 12-month wastage rate trend chart with 9.61% baseline line • Recovery pathway breakdown today • Urgent at-risk units list	• Connected to analyticsApi.getWasteAnalytics and inventory summary/unit endpoints
10. Camp Planning	• 6-Month collection timeline • Expected collection need (3,240–4,120 units) • Sensitivity analysis & WHO xMart seasonal index (1.83) • Bangladesh analogue evidence ($r = +0.970$)	• View/Hide Assumptions toggle • Adjust Plan button opens interactive parameters drawer modal and updates targets
11. Login	• 3D Earth Globe ('react-globe.gl') with hospital stock pins across India • Clean login card	• Continue as demo user single-click login CTA • Sign in with Google OAuth fallback

3. Technical Architecture & Two-Track Data Principle

PlateletIQ follows a strict **Two-Track Data Architecture** to guarantee clinical data provenance. The ML demand model runs strictly on verified hospital time-series data, while regional network transfers leverage scraped public availability.

Track	Data Source	Screens Covered	Provenance Badge
Track 1: ML Demand & Decision Engine	German Hospital Dataset (Aachen 2008–2018, 4,018 days) + WHO India Index	Overview, Forecast, Actions, Inventory, Data Setup, Camp Planning	MODEL / SIMULATED / DEMO
Track 2: Regional Network Scraper	eRaktKosh National Portal Scraper Database (756 hospitals across 32 districts)	Network, Transfers & Recovery	EXTERNAL

4. Ground Truth Parameters & Verification Results

Metric / Parameter	Value	Provenance Badge
PlateletIQ Forecast MASE	0.734	MODEL
Naive Baseline MASE	0.993	MODEL
Schilling Benchmark MASE	0.746	MODEL
Optimized Wastage Rate	3.25% (vs 9.61% baseline)	SIMULATED
Optimized Shortage Rate	3.04% (vs 6.73% baseline)	SIMULATED
Annual Cost Savings	Rs 7,20,000 / hospital	SIMULATED
National SDP Stock Share	4.7% (23/32 districts zero SDP)	EXTERNAL
November Seasonal Index	1.83 (+83% peak demand)	EXTERNAL

5. Teammate Quick-Start & Local Execution

To run and test the complete application locally:

1. **Backend Server Launch:** Open terminal in backend/ and run:
`python -m uvicorn main:app --port 8000`
2. **Frontend Server Launch:** Open terminal in frontend/ and run:
`npm run dev`
3. **Browser Access:** Open **`http://localhost:5173/`** in your browser and click *Continue as demo user*.
4. **Backend API Documentation:** Interactive OpenAPI docs are available at **`http://127.0.0.1:8000/docs`**.